New Horizons®
Centro de Capacitación

Curso

Desarrollo Full Stack con C# y Angular

Docente: Edwin Barrientos



New Horizons®
Centro de Capacitación

Sesión 11 — Módulo 3:

Consumo de API REST y RxJS



Objetivo del módulo: conectar el frontend Angular 18 de forma integral con el backend ASP.NET Core Web API de C#, dominando provideHttpClient, servicios inyectables tipados, operadores de RxJS, interceptores HTTP para RFC 7807 y reactividad con Signals.



New Horizons®
Centro de Capacitación

Agenda

- Configuración moderna de HttpClient con provideHttpClient y withFetch
- Servicios inyectables y tipado estricto con DTOs de C#
- Operadores clave de RxJS: map, switchMap, tap, catchError y finalize
- Manejo centralizado de errores: Interceptor RFC 7807 ProblemDetails
- Spinner global con Signals e integración CRUD en tiempo real



New Horizons®
Centro de Capacitación

Sesión 1:

Consumo de la API REST de C# con HttpClient y RxJS

La integración Full Stack en vivo

New Horizons
Centro de Capacitación

🍽️ **Piensa en un restaurante.** Angular es el mesero que toma el pedido, ASP.NET Core es la cocina y SQL Server la despensa. HttpClient es la comanda que viaja entre la mesa y la cocina.

Angular 18 → **ASP.NET Core 8 Web API** → **SQL Server**

HttpClient Nativo

Cliente HTTP basado en fetch API altamente eficiente y asíncrono.

Piénsalo así: *Es la comanda: lleva el pedido a la cocina y trae el plato de vuelta.*

RxJS & Signals

Observables para peticiones HTTP y Signals para estado reactivo local.

Piénsalo así: *El Observable es el delivery en camino; el Signal es el plato ya servido en la mesa.*

Seguridad & CORS

Políticas de origen cruzado configuradas en el pipeline de C#.

Piénsalo así: *El portero de la cocina: solo entran meseros de la lista (localhost:4200).*

Configuración moderna en app.config.ts

New Horizons®
Centro de Capacitación

app.config.ts

```ts
import { ApplicationConfig } from "@angular/core";
import { provideHttpClient, withFetch, withInterceptors }
  from "@angular/common/http";
import { loadingInterceptor }
  from "./interceptors/loading.interceptor";
import { errorInterceptor }
  from "./interceptors/error.interceptor";

export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withFetch(),
      withInterceptors([loadingInterceptor, errorInterceptor])
    )
  ]
};
```



Una regleta de enchufes

provideHttpClient es la regleta; withFetch y withInterceptors son los adaptadores que le conectas.

1 **provideHttpClient()**

Reemplaza al antiguo HttpClientModule: configuración standalone, sin NgModule.

2 **withFetch()**

Usa la Fetch API nativa del navegador: más ligera y compatible con SSR.

3 **withInterceptors([...])**

Los interceptores se ejecutan en orden: primero el loader, luego el de errores.

Variables de entorno con environment.ts

New Horizons®
Centro de Capacitación

environment.ts

```ts
// src/environments/environment.ts
export const environment = {
  production: false,
  apiUrl: "http://localhost:5000/api"
};

// src/environments/environment.prod.ts
export const environment = {
  production: true,
  apiUrl: "https://api.empresa.com/api"
};
```

1
2



Un GPS con dos destinos

El viaje (tu código) es el mismo; solo cambia la dirección según sea ensayo o función real.

1 **DEV** `ng serve`

`localhost:5000/api`

2 **PROD** `ng build`

`api.empresa.com/api`

💡 Angular cambia un archivo por otro al compilar gracias a `fileReplacements` en angular.json. Tus servicios solo importan `environment`.

El servicio SolicitudHttpService

solicitud-http.service.ts

```ts
import { Injectable, inject } from "@angular/core";
import { HttpClient, HttpParams } from "@angular/common/http";
import { Observable } from "rxjs";
import { environment } from "../../environments/environment";

@Injectable({ providedIn: "root" })
export class SolicitudHttpService {
  private http = inject(HttpClient);
  private baseUrl = `${environment.apiUrl}/solicitudes`;

  obtenerPaginado(p: number = 1, t: number = 10)
      : Observable<PagedResult<Solicitud>> {
    const params = new HttpParams()
      .set("pagina", p).set("tamanoPagina", t);
    return this.http.get<PagedResult<Solicitud>>(this.baseUrl,
      { params });
  }
}
```

La central telefónica

Los componentes no marcan directo a la API: llaman a la central, que sabe el número y el protocolo.

1 providedIn: "root"

Una sola central para toda la app (singleton).

2 baseUrl desde environment

El número se arma una vez; nadie lo escribe a mano.

3 HttpParams

Arma el query string sin concatenar strings.

4 get<PagedResult<Solicitud>>

TypeScript ya conoce la forma de la respuesta.

```
GET /api/solicitudes?pagina=1&tamanoPagina=10
```

Operaciones CRUD tipadas

New Horizons
Centro de Capacitación

crud-methods.ts

```ts
// Métodos POST, GET por ID, PUT y DELETE conectados al Web API:
obtenerPorId(id: number): Observable<Solicitud> {
  return this.http.get<Solicitud>(`${this.baseUrl}/${id}`);
}

crear(dto: CrearSolicitudDto): Observable<Solicitud> {
  return this.http.post<Solicitud>(this.baseUrl, dto);
}

actualizar(id: number, dto: ActualizarSolicitudDto)
    : Observable<void> {
  return this.http.put<void>(`${this.baseUrl}/${id}`, dto);
}

eliminar(id: number): Observable<void> {
  return this.http.delete<void>(`${this.baseUrl}/${id}`);
}
```



Cuatro ventanillas del banco

Cada verbo HTTP es una ventanilla distinta: consultar, abrir, modificar o cerrar una cuenta.

1	GET	/{id}	200 · Solicitud
2	POST	/	201 · Created
3	PUT	/{id}	204 · No Content
4	DELETE	/{id}	204 · No Content

RxJS en acción: switchMap para evitar carreras

New Horizons
Centro de Capacitación



Como cambiar de canal: si pasas del 5 al 7, la TV deja de cargar el 5. switchMap hace lo mismo: cancela la petición vieja cuando llega una nueva.

El usuario escribe en el buscador…

“c”	“co”	“com”
✗ cancelada	✗ cancelada	✓ respuesta

ejemplo-busqueda.ts

```
this.busqueda.valueChanges.pipe(
  debounceTime(300),
  switchMap(t => this.api.buscar(t))
).subscribe(r => this.resultados.set(r));
```

Búsqueda en Vivo

Combina debounceTime(300) con switchMap para no saturar al backend.

Esperas a que el usuario deje de teclear.

Sin Memory Leaks

Cancela el observable interno si el componente se destruye.

Nada queda escuchando en segundo plano.

Resultado Fresco

Garantiza que la respuesta visible corresponda siempre a la última consulta.

Siempre ves el canal que elegiste al final.

LoadingInterceptor: Spinner global

New Horizons®
Centro de Capacitación

loading.interceptor.ts

```ts
import { HttpInterceptorFn } from "@angular/common/http";
import { inject } from "@angular/core";
import { finalize } from "rxjs";
import { LoadingService } from "../services/loading.service";

export const loadingInterceptor: HttpInterceptorFn =
  (req, next) => {
    const loader = inject(LoadingService);
    loader.mostrar(); // Incrementa peticiones activas

    return next(req).pipe(
      finalize(() => loader.ocultar()) // éxito o error
    );
  };
```

1
2
3

El contador del peaje

Cada auto que entra suma, cada uno que sale resta. La luz de “ocupado” se apaga solo cuando no queda nadie.

1 — Función, no clase

HttpInterceptorFn: recibe la petición y el siguiente eslabón.

2 — mostrar() = entra un auto

El contador de peticiones activas sube en 1.

3 — finalize() = sale el auto

Corre siempre: éxito, error o cancelación.

0	1	2	1	0

Peticiones activas: el spinner se ve mientras el número sea mayor que 0.

ErrorInterceptor: Mapeo RFC 7807

New Horizons®
Centro de Capacitación

error.interceptor.ts

```ts
import { HttpInterceptorFn } from "@angular/common/http";
import { inject } from "@angular/core";
import { catchError, throwError } from "rxjs";
import { MatSnackBar } from "@angular/material/snack-bar";

export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const snack = inject(MatSnackBar);
  return next(req).pipe(
    catchError(err => {                                          // 1
      const msg = err.error?.detail || err.error?.title          // 2
                  || "Error de conexión";
      snack.open(msg, "Cerrar", { duration: 4000 });
      return throwError(() => err);                              // 3
    })
  );
};
```

El traductor de la embajada

El backend habla ProblemDetails (RFC 7807); el usuario necesita una frase clara en pantalla.

ProblemDetails · 400

```
{ "title": "Solicitud inválida",
  "status": 400,
  "detail": "El monto excede el tope." }
```

El monto excede el tope. CERRAR

💡 ① atrapa · ② traduce (detail → title → genérico) · ③ re-lanza con `throwError` para que el componente decida.

Puente reactivo: toSignal() en Angular 18

New Horizons
Centro de Capacitación

to-signal.ts

```ts
import { Component, inject } from "@angular/core";
import { toSignal } from "@angular/core/rxjs-interop";
import { SolicitudHttpService }
  from "../../services/solicitud-http.service";

export class SolicitudesListComponent {
  private api = inject(SolicitudHttpService);

  // Convierte el Observable de la API en un Signal
  // sin suscripción manual:
  solicitudes = toSignal(this.api.obtenerPaginado(1, 50),
                         { initialValue: [] });
}
```

1 2

El adaptador de viaje

Conecta el enchufe RxJS (Observable) al tomacorriente Signals, sin pelar cables (subscribe/unsubscribe).

`Observable` › `toSignal()` › `Signal`

1 Se suscribe y se limpia solo

Atado al ciclo de vida del componente: cero fugas.

2 initialValue

Valor mientras llega la respuesta: nunca undefined.

En el template se lee como función: `solicitudes()`. Nada de `| async`.

Configuración de CORS en ASP.NET Core

New Horizons
Centro de Capacitación

Control migratorio: el navegador es el agente de migraciones y bloquea a cualquier origen que el backend no tenga en su lista de visas aprobadas.

Program.cs

```
1  builder.Services.AddCors(o => o.AddPolicy("AllowAngularApp",
       p => p.WithOrigins("http://localhost:4200")
2            .AllowAnyHeader()
             .AllowAnyMethod()));

   var app = builder.Build();
3  app.UseCors("AllowAngularApp");    // antes de UseAuthorization
   app.UseAuthorization();
```

④ Preflight: la pregunta antes del viaje

🌐 → OPTIONS /api/solicitudes — **204 No Content**

🌐 → POST /api/solicitudes — **201 Created**

1. **AddCors en Program.cs**

   Declara la política AllowAngularApp con WithOrigins("http://localhost:4200").

2. **AllowAnyHeader & Method**

   Permite Content-Type, Authorization y verbos POST, PUT, DELETE.

3. **UseCors(Policy)**

   Debe ir antes de UseAuthorization en el pipeline de middlewares.

4. **Preflight (OPTIONS)**

   El navegador pregunta primero; ASP.NET Core responde 204 No Content.

Guardado real en Base de Datos desde el Formulario

New Horizons
Centro de Capacitación

guardar-real.ts

```
guardar(): void {
  if (this.formulario.invalid) return;
  const dto = this.formulario.value;

  this.httpService.crear(dto).subscribe({
    next: (creada) => {
      this.snackBar.open(
        `¡Solicitud ${creada.codigo} registrada!`, "OK",
        { duration: 3000 });
      this.router.navigate(["/solicitudes"]);
    }
  });
}
```

Una carta certificada

La entregas (subscribe), esperas el acuse de recibo (next) y recién entonces celebras y sigues.

1 Guardia de entrada

Si el formulario es inválido, la carta ni sale.

2 Sin subscribe no hay viaje

El Observable es perezoso: sin suscripción no hay petición.

3 next = acuse de recibo

Mostrar el código generado y volver a la lista.

¡Solicitud SOL-000123 registrada! OK

¿Y si falla? No hace falta un bloque error: el ErrorInterceptor ya muestra el mensaje.

Paginación en Servidor: Skip y Take

New Horizons
Centro de Capacitación

La biblioteca no te entrega todos los libros: te da el estante que pediste (la página) y te dice cuántos libros hay en total para armar el índice.

Filas por página: 10 ▾ **11 – 20 de 10,000** ‹ ›

```
.Skip((pagina - 1) * tamanoPagina)
.Take(tamanoPagina)
```

paginaActual

Número de página solicitado (1, 2, 3...).

Piénsalo así: *El número de estante.*

tamanoPagina

Cantidad de filas por página (5, 10, 20).

Piénsalo así: *Cuántos libros caben por estante.*

totalItems

Total de registros en base de datos para calcular el paginador.

Piénsalo así: *El catálogo completo, para saber cuántos estantes hay.*

Actualización de UI: Optimista vs Pesimista

New Horizons®
Centro de Capacitación

Actualización Optimista

Como el "me gusta" de una red social: el corazón se pinta al instante; si falla, se despinta.

- ✓ La interfaz se actualiza al instante sin esperar la red
- ✓ Excelente para likes, favoritos o listas rápidas
- ✗ Requiere rollback si la petición HTTP falla en el backend

Secuencia

Clic → UI ✓ → API… → ↩ rollback si falla

Actualización Pesimista

Como una transferencia bancaria: no ves "enviado" hasta que el banco confirma.

- ✓ La UI se actualiza solo tras recibir respuesta 200/201
- ✓ Obligatorio en transacciones financieras y pagos
- ✓ Garantiza consistencia absoluta de estados en C#

Secuencia

Clic → API… → 201 → UI ✓

⚖️ **Regla práctica:** si equivocarse es barato, sé optimista; si cuesta dinero, sé pesimista.

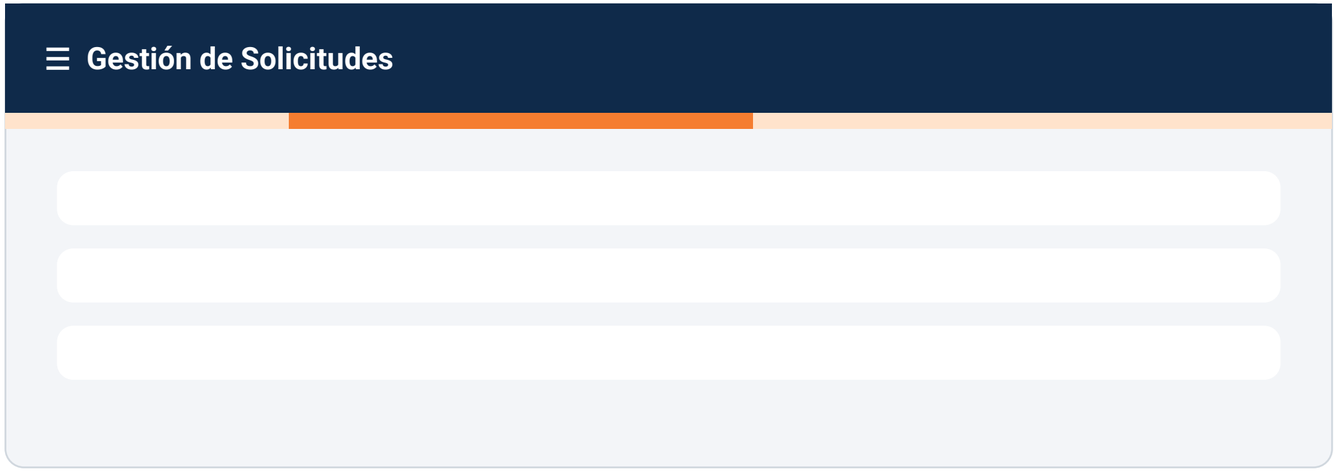
Barra de progreso global en el Shell

New Horizons
Centro de Capacitación

progress-bar.html

```
<!-- En la cabecera del ShellComponent: -->
<mat-progress-bar
  *ngIf="loader.estaCargando()"
  mode="indeterminate"
  color="accent">
</mat-progress-bar>
```

Así se ve en el Shell



El turnero del banco

Te avisa que algo está pasando sin bloquear la pantalla ni obligarte a esperar mirando un modal.

1 *ngIf lee un Signal

estaCargando() se evalúa en cada cambio; sin subscribe.

2 mode="indeterminate"

No sabemos cuánto falta: la barra solo “se mueve”.

En el LoadingService: `estaCargando = computed(() => this.activas() > 0)`. Un solo lugar del Shell muestra el progreso de toda la app.

Validación de la conexión con Network DevTools

New Horizons®
Centro de Capacitación

Network es el rastreo del courier: ves cada paquete que sale y llega, su estado, su tipo y sus cabeceras. Si no aparece aquí, no ocurrió.

Método	URL	Estado	Tipo
OPTIONS	/api/solicitudes	204	preflight
GET	/api/solicitudes?pagina=1	200	json
POST	/api/solicitudes	201	json
POST	/api/solicitudes	400	problem+json

Filtra por **Fetch/XHR**, abre una fila y revisa **Headers** (Access-Control-Allow-Origin, Location), **Payload** (lo que envió Angular) y **Response** (lo que contestó C#).

- ✓ GET /api/solicitudes devuelve cabeceras CORS y array de elementos.
- ✓ POST /api/solicitudes devuelve 201 Created con cabecera Location.
- ✓ Payloads inválidos devuelven 400 Bad Request con ProblemDetails RFC 7807.
- ✓ La base de datos de SQLite/SQL Server conserva los cambios entre reinicios.

El ciclo de vida de una petición HTTP

New Horizons
Centro de Capacitación

1

Disparo

El mesero lleva la comanda

- ✓ Componente invoca httpService
- ✓ LoadingInterceptor activa spinner

Visto en: *Servicio tipado · interceptores*

2

Procesamiento C#

La cocina prepara el plato

- ✓ Validación FluentValidation
- ✓ Persistencia con Unit of Work

Visto en: *CORS · ProblemDetails · Skip/Take*

3

Respuesta & UI

El plato llega a la mesa

- ✓ finalize() apaga spinner
- ✓ Signals actualizan la tabla

Visto en: *toSignal · barra de progreso*

Si la cocina rechaza el pedido (400), el **ErrorInterceptor** traduce el motivo y el spinner igual se apaga.

Checklist de calidad en consumo de APIs

New Horizons®
Centro de Capacitación

Checklist del mecánico: nadie entrega un auto sin revisarla; tú no despliegas un cliente HTTP sin ella.

✓ **Centraliza las URLs base en environment.ts.**	→ *Cambiar de ambiente es un build, no un refactor.*
✓ **Tipa cada petición y respuesta, sin `any`.**	→ *El compilador detecta DTOs desalineados.*
✓ **Usa interceptores funcionales.**	→ *Las clases de interceptores son el estilo antiguo.*
✓ **Apaga los spinners siempre con finalize().**	→ *Nunca más una pantalla “cargando” eterna.*
✓ **Verifica CORS en el backend antes de desplegar.**	→ *Producción tiene otro origen que localhost.*

Errores comunes en HttpClient y CORS

New Horizons
Centro de Capacitación

✗ Evita esto	✓ Haz esto
Hardcodear http://localhost:5000 en componentes	Inyectar la URL desde environment.apiUrl
Olvidar suscribirse (la petición nunca sale)	Usar toSignal() o suscribirse en el momento adecuado
Suscripciones anidadas tipo callback hell	Encadenar peticiones con switchMap y operadores RxJS
Ignorar el error y dejar la pantalla congelada	Mapear errores con ErrorInterceptor y MatSnackBar
Olvidar withFetch() en proyectos modernos	Habilitar withFetch() para rendimiento optimizado

Buenas prácticas de integración de microservicios

New Horizons®
Centro de Capacitación



Frontend y backend son pareja de baile: si uno cambia el paso (un DTO), el otro tiene que enterarse antes de pisarlo.



Modelos sincronizados

Mantén las interfaces de modelos sincronizadas con los DTOs de C#.

Un cambio en C# = un cambio en la interfaz TS



Estado local con Signals

Usa signals para almacenamiento local de colecciones en memoria.

La lista vive en un signal, no en variables sueltas



Avisos breves y claros

Emite notificaciones breves y claras al usuario ante caídas de red.

“Sin conexión. Reintenta en unos segundos.”



Prueba en paralelo

Prueba peticiones desde Swagger y desde Angular en paralelo.

Si Swagger funciona y Angular no: es el frontend o CORS

Resumen de la sesión

New Horizons®
Centro de Capacitación

🌐 HttpClient

provideHttpClient con withFetch y tipado.

≈ La comanda

💻 Environment

URLs base centralizadas para dev y prod.

≈ El GPS

⚡ RxJS

Operadores switchMap, finalize y toSignal.

≈ Cambiar de canal

🛡 Error RFC 7807

ProblemDetails mapeado hacia MatSnackBar.

≈ El traductor

⏳ Loader Global

Indicador de progreso gestionado con Signals.

≈ El contador del peaje

💾 Persistencia Real

CRUD conectado y verificado en base de datos.

≈ La carta certificada

PREGUNTAS...

New Horizons®
Centro de Capacitación

1. ¿Cómo reemplaza provideHttpClient(withFetch()) al antiguo HttpClientModule?
2. ¿Por qué es crucial tipar cada respuesta HTTP con interfaces TypeScript espejo de C#?
3. ¿Cómo captura un HttpInterceptor los errores 400 de FluentValidation y los transforma para la UI?
4. ¿Cuál es la diferencia entre manejar el estado con RxJS BehaviorSubject vs Signals nativos?
5. ¿Cómo resolvemos el problema de CORS entre localhost:4200 y localhost:5000?

New Horizons
Centro de Capacitación

Conclusiones

HttpClient moderno

provideHttpClient con withFetch optimiza la transferencia de datos y el soporte SSR.

Contratos tipados

El tipado estricto con DTOs evita discrepancias silenciosas entre frontend y backend.

Interceptores desacoplados

Los interceptores funcionales desacoplan el manejo de carga (loader) y errores del código de vistas.

Errores que se entienden

El mapeo de ProblemDetails (RFC 7807) permite presentar al usuario los fallos de validación exactos de C#.

Lo mejor de dos mundos

toSignal() une lo mejor de RxJS (flujos asíncronos) con la reactividad predecible de Angular 18.

GRACIAS